SOFTWARE ENGINEERING
TEAM PROJECT

Deadlines:

- **Sprint Starts:** 10/23/26
- **Presentation**: during class 11/13/26 and 11/16/26

Directions:

- Fork my project on GitHub to create your own project for your team
- Create a "project" on GitHub for your repo and link your repo to it
- Create Milestones for your project and attach each issue/user story to a milestone
- Create meaningful labels and use them for your issues/stories.
- You will need to figure out which (if any) user stories from the product backlog you want in this sprint. You can also add your own user stories.
- You may have to write some stories covering any polish or bug fixes from the original project that need to be worked on. Feel free to treat these as normal stories and point them just like the rest. Normally the result of a sprint is a polished product, however it is not uncommon to receive a project that is unpolished and be asked to implement features for it.
- Play planning poker to point the stories.
- Use these points to ensure even workload of all members. Remember team members assign work to themselves. This gives developers a higher sense of ownership over the features they have personally selected to work on. Just make sure at the end of the sprint planning meeting, that everyone has roughly the same number of points.
- Make sure there is enough work to keep everyone busy
- Figure out how you will communicate with your team during the sprint (Discord, Hangouts, Whats App, etc.)
- Most class periods we will have 10 mins dedicated to daily standups for all teams

Retrospective document (Submit the following information in one .pdf file on Moodle):

- List of team members (first and last names of each person).
- URL to your GitHub repo.
    - I should have permission to access both the repo and the linked project.
- Answer the following questions:
    - What was your estimated vs. your actual velocity for your sprint? In other words, how many story points did you plan to accomplish and how many did you actually accomplish? Did you finish everything for your sprint? Is your product potentially shippable? Did you get all the core/most important stories accomplished? If you were to do another sprint, would you increase or decrease the amount of story points in your planned workload for that sprint (i.e. would you attempt to increase or decrease your velocity)?
    - Give me a table showing the number of story points each team member individually accomplished during the sprint. If help was given on a story, count the points for all team members involved. Only count the stories that were finished though.
    - Tell me how many total Git commits each team member did (also put this in a table). Doesn't matter which branch, I just want to know the total number of commits for the entire sprint.

- Overall, what kinds of things could you do to increase your velocity if you were to do another sprint? Would you even follow Scrum again or would you make some changes and perhaps introduce some concepts from other methodologies? Could concepts from Kanban (with WIP limits, swimlanes, etc) or Extreme Programming (Pair programming, collective ownership, etc.) help?

Grading:

Effort Multiplier

On Wednesday Nov 4th, I will ask about team member effort by giving everyone a form to fill out. This will be considered for the final grade of this project.

On Friday Nov 13th, I will ask you again about team member effort (same form). Both of these forms will be taken into account for grading.

I will also use the reported number of GitHub commits and the report number of story points completed by each team member.

All of these go into an effort multiplier. It is very simple. If the team score (described below) is a 90% but individually you did half of the work of other team members, you will receive 90% * 0.5 or 45% on the project. This is just an example and actual team score may vary, but hopefully you get the idea. Want to receive the maximum number of points that the team as a whole earned? Then you need to match or exceed the amount of effort given by your teammates.

Team Score

On the presentation day(s), your group will present their project. I will grade you on what you have done to improve the project since I last touched it (i.e. from the point at which you forked it).
I care less about the total number of story points accomplished by your team and more about what that actually means in terms of improvements to the project. I am acting as an end-user in this and what I can't see, I can't grade. So if you spent 5 hours refactoring the code for a 10 point story, none of that is going to matter in the end. The points should really only be used to ensure equal work among team members as the end-user rarely, if ever, sees this.

In general, these are the things I am looking at:

- New features added
- Stability of overall project
  - It should feel like a PSP and not be buggy
- Pre-existing bugs/issues from my code that were fixed
  - Point these out during your presentation
- Sensibility of design decisions
  - I don't expect anyone to be an expert in sound design (that is not the purpose of this course), however this is a software engineering project and every decision has pros and cons
  - The designs you make when designing how your software operates should be easy for an end-user to understand and to use (just like how your coding design decisions should be easy for another developer to understand and use)
  - Feel free to have friends/family try your sound designer out for feedback